

Lista de Exercícios #6 – Métodos de Busca

Valor: 3 pontos

Esta lista de exercícios é baseada nos assuntos discutidos em aula e no conteúdo dos seguintes slides disponibilizados no CANVAS:

- 15 - Busca sem informação
- 16 - Busca com informação
- 17 - Busca competitiva

Orientações gerais:

- A entrega deverá ser realizada em um arquivo em formato PDF na respectiva tarefa do CANVAS.

Questão 01. Sobre os principais conceitos de métodos de busca, responda:

1. Um agente de resolução de problemas precisa ser capaz de formalizar qualquer situação antes de buscar uma solução. Quais são os cinco componentes exigidos para definir um problema de busca em IA? Defina e ilustre cada um com o exemplo do 8-puzzle.
 - Estado inicial: configuração de partida do agente. No 8-puzzle, é qualquer arranjo das oito peças no tabuleiro 3×3, por exemplo, peças 1-8 em posições embaralhadas com o espaço vazio em algum quadrado.
 - Conjunto de ações: operações disponíveis a cada estado. No 8-puzzle, as ações são os quatro movimentos possíveis do espaço vazio: esquerda, direita, acima e abaixo (nem todas disponíveis nas bordas e cantos).
 - Modelo de transição: função que, dado um estado e uma ação, retorna o estado resultante. No 8-puzzle, deslizar uma peça adjacente para o espaço vazio gera um novo arranjo de peças.
 - Teste do objetivo: verifica se um estado é o estado-alvo. No 8-puzzle, consiste em comparar a configuração atual com a configuração-alvo (por exemplo, peças ordenadas de 1 a 8 com o espaço no canto inferior direito).
 - Custo da solução: medida de qualidade de um caminho. No 8-puzzle, cada ação teria custo 1, portanto o custo total é igual ao número de movimentos realizados; a solução ótima minimiza esse número.

2. Compare os algoritmos de Busca em Largura (BFS) e Busca em Profundidade (DFS) nos critérios de completude e otimalidade. Descreva um cenário ou problema em que a Busca em Profundidade poderia falhar (não terminar).

A Busca em Largura (BFS) é completa (sempre encontra a solução se existir e a profundidade for finita) e ótima (se os custos das arestas forem iguais). Já a Busca em Profundidade (DFS) não é completa nem ótima. A DFS pode falhar se o espaço de busca do problema contiver caminhos infinitos ou se houver ciclos (loops). Nesse caso, o algoritmo ficará descendo infinitamente sem nunca retornar e testar o objetivo.

3. A Busca de Custo Uniforme (equivalente ao algoritmo de Dijkstra) é uma generalização da BFS para grafos com custos variáveis nas arestas. Explique por que a BFS simples não é adequada quando as ações têm custos diferentes. Descreva como a Busca de Custo Uniforme garante encontrar a solução de menor custo. Qual estrutura de dados é fundamental para sua eficiência?

A BFS simples trata todas as ações como equivalentes, expandindo nós por nível (expande todos os filhos primeiro, e assim sucessivamente). Quando as ações têm custos diferentes, a solução com menos passos pode não ser a de menor custo total. Por exemplo, em um grafo com três caminhos $A \rightarrow B$ de 1 passo, a BFS encontraria o primeiro independentemente de seus custos serem 1, 10 ou 100. A Busca de Custo Uniforme (Dijkstra) corrige isso expandindo sempre o nó de menor custo acumulado na fronteira.

Otimalidade: ao expandir um nó n com custo $g(n)$, qualquer outro caminho até n teria custo maior ou igual (pois todos os custos são não negativos). Portanto, o primeiro caminho até o objetivo encontrado pela BCU é necessariamente o de menor custo.

Estrutura de dados fundamental: fila de prioridades (heap), que permite inserção em $O(\log n)$ e remoção do mínimo em $O(\log n)$, tornando a busca eficiente.

4. O que é uma função heurística $h(n)$ e como ela diferencia a Busca com Informação da Busca sem Informação? Cite pelo menos um exemplo de heurística abordada no material (por exemplo, no mapa da Romênia ou no 8-puzzle).

A função heurística $h(n)$ é um valor numérico que estima a “distância” ou custo do nó atual até o estado objetivo. Ela permite que a busca seja direcionada (usando informações sobre quão promissor é um estado), diferente da busca sem informação que explora o espaço de busca de maneira bruta.

Exemplos: No trajeto da Romênia, a $h(n)$ é a *distância em linha reta de uma cidade até Bucareste*. No 8-puzzle, pode ser a *quantidade de peças fora do lugar* ou a *distância de Manhattan*.

5. Tanto a Busca Gulosa quanto o A* utilizam funções heurísticas, mas com estratégias diferentes. Compare os dois algoritmos quanto às suas funções de avaliação $f(n)$, e demonstre com um exemplo (como o problema da viagem na Romênia) por que a Busca Gulosa pode encontrar soluções subótimas enquanto o A*, com heurística admissível, sempre encontra a solução ótima.

Funções de avaliação:

- Busca Gulosa: $f(n) = h(n)$ – considera apenas a estimativa do custo até o objetivo, ignorando o custo já percorrido.
- A*: $f(n) = g(n) + h(n)$ – combina o custo real já percorrido $g(n)$ com a estimativa $h(n)$.

Exemplo: viagem na Romênia (Arad → Bucareste), heurística = distância em linha reta.

Busca Gulosa expande: Arad → Sibiu → Fagaras → Bucareste. Caminho com custo $140+99+211 = 450$ km. Não é ótimo.

A* expande um conjunto maior de nós mas encontra: Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucareste, com custo $140+80+97+101 = 418$ km (a solução ótima).

A Busca Gulosa falha porque, ao ignorar o custo acumulado $g(n)$, ela pode se comprometer com um caminho que parece promissor localmente (estar “perto” do objetivo em linha reta), mas que acumula custos reais maiores. Já o A* com heurística admissível garante otimalidade porque $f(n)$ é um limite inferior do custo real de qualquer solução passando por n . Dessa forma, o A* nunca descarta um caminho potencialmente ótimo antes de explorá-lo.

6. Você está desenvolvendo um agente inteligente para três cenários distintos: (a) encontrar o caminho mais curto em um mapa de cidades onde as distâncias em linha reta são conhecidas; (b) jogar xadrez contra um oponente humano com limite de tempo por jogada; (c) resolver um labirinto desconhecido sem nenhuma informação prévia sobre o ambiente. Para cada cenário, indique qual estratégia de busca seria mais adequada (dentre BFS, DFS, Busca com Aprofundamento Iterativo, A*, MINIMAX com poda alfa-beta) e justifique sua escolha com base nas propriedades de cada algoritmo estudado.

Cenário (a): encontrar o caminho mais curto em mapa de cidades com distâncias em linha reta conhecidas → Algoritmo recomendado: A*

Justificativa: a distância em linha reta entre cidades é uma heurística admissível e consistente (nunca superestima a distância rodoviária real). O A* usa essa informação para guiar a busca diretamente ao objetivo, expandindo muito menos nós do que a Busca de Custo Uniforme (Dijkstra) sem sacrificar a otimalidade.

Cenário (b): jogar xadrez contra um humano com limite de tempo por jogada. → Algoritmo recomendado: MINIMAX com poda alfa-beta (poderia ser melhorada com profundidade limitada + função de avaliação heurística)

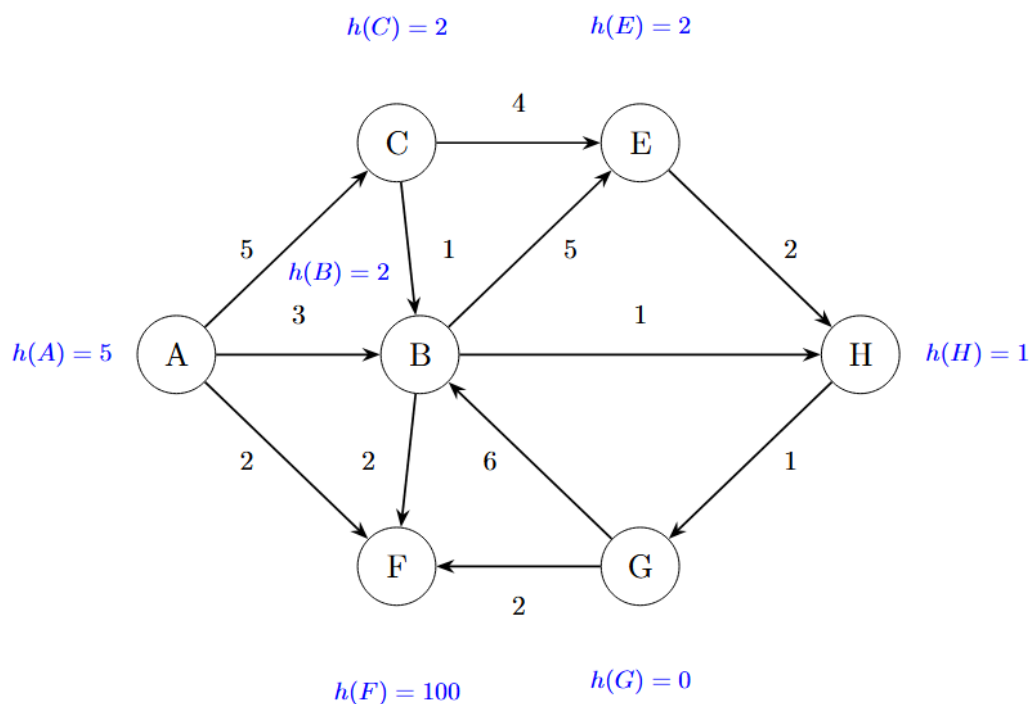
Justificativa: o xadrez é um jogo de soma zero, determinístico, de dois agentes adversários, que é o cenário clássico do MINIMAX.

Melhoria: Como a árvore completa é inviável ($\approx 10^{154}$ nós), combina-se a poda alfa-beta (reduz a complexidade para $O(b^{(m/2)})$) com um limite de profundidade e uma função de avaliação heurística para os nós não-terminais. A ordenação de movimentos (capturas primeiro) maximiza a eficiência da poda dentro do tempo disponível.

Cenário (c): resolver labirinto desconhecido sem nenhuma informação prévia → Algoritmo recomendado: Busca com Aprofundamento Iterativo (ou BFS se a memória permitir)

Justificativa: sem informação sobre o ambiente, nenhuma heurística está disponível (ou seja, busca com informação não se aplica). Não há adversário, então MINIMAX também não se aplica. Nesse caso, a Busca com Aprofundamento Iterativo seria ideal porque: (1) é completa, i.e., encontra a saída se ela existir; (2) é ótima em número de passos (encontra o caminho mais curto); A BFS também seria ótima e completa, mas exigiria mais memória, o que poderia ser um problema em labirintos com muitas bifurcações.

Questão 02. Considere o problema de busca definido pelo seguinte grafo, onde A é o estado inicial e G é o estado objetivo, e o custo de cada ação é indicado na aresta correspondente.

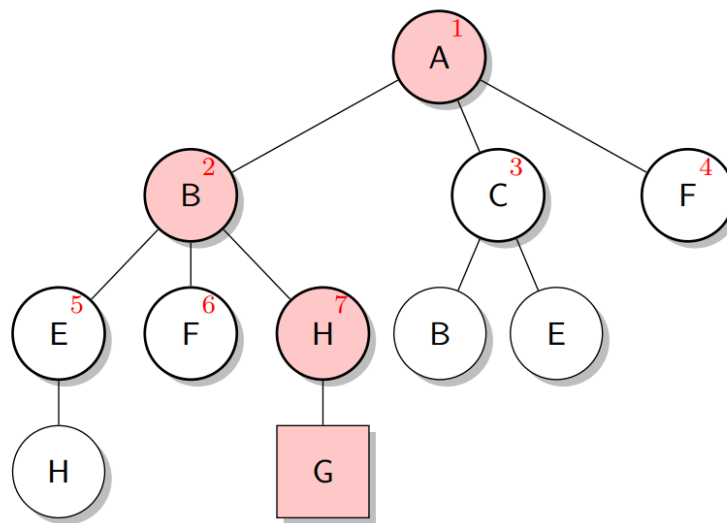


Resolva o problema utilizando as seguintes estratégias de busca:

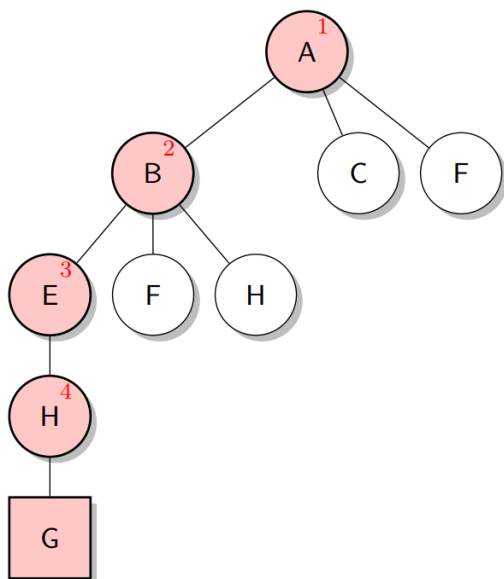
- Busca em largura (BFS)
- Busca em profundidade (DFS)
- Busca de custo uniforme
- Busca gulosa (o valor da heurística para cada nó n é dado por $h(n)$)
- A* (o valor da heurística para cada nó n é dado por $h(n)$)

Para cada estratégia, mostre a árvore de busca final, indicando o caminho encontrado de A a G e o custo da solução total. Considere a ordem de visitação em ordem alfabética.

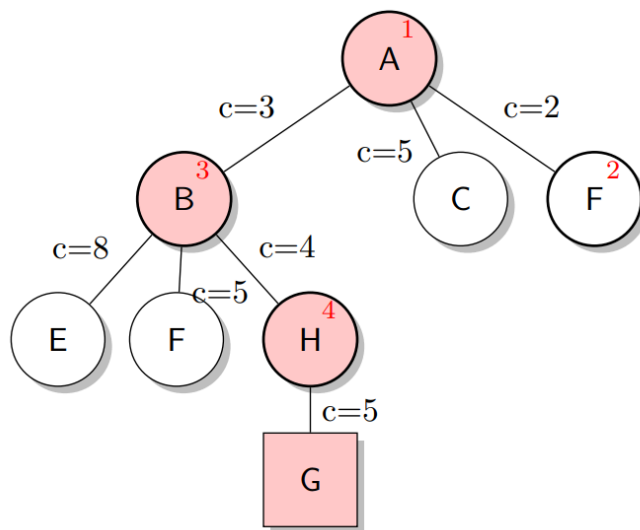
BFS (ordem dos nós visitados em vermelho): custo $3+1+1 = 5$



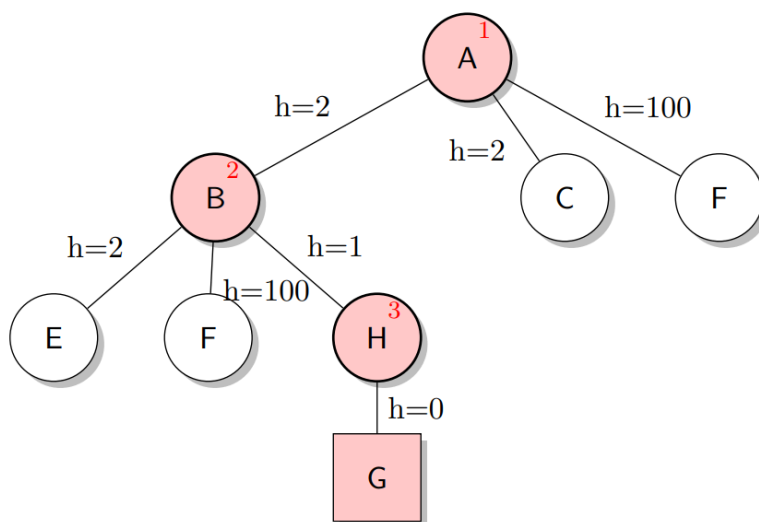
DFS (ordem dos nós visitados em vermelho): custo $3+5+2+1 = 11$



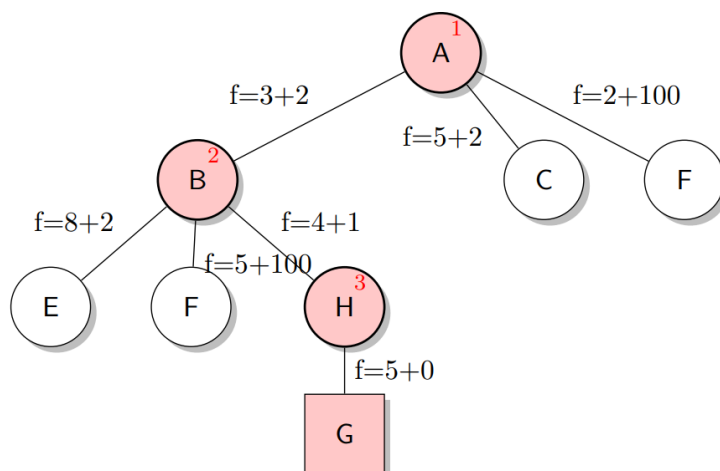
Busca com Custo Uniforme (ordem dos nós visitados em vermelho): custo $3+1+1 = 5$



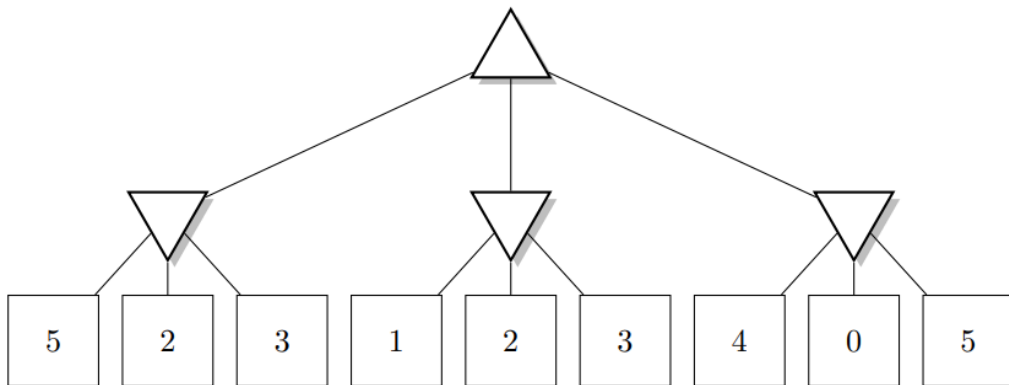
Busca Gulosa (ordem dos nós visitados em vermelho): custo $3+1+1 = 5$



A* (ordem dos nós visitados em vermelho): custo $3+1+1 = 5$



Questão 03. Considere a seguinte árvore representando um jogo de soma zero, onde os triângulos apontando para cima são nós MAX, os triângulos apontando para baixo são nós MIN e os quadrados são nós objetivos (terminais) com o valor correspondente da função de utilidade para o jogador MAX.



Aplique o algoritmo MINIMAX para encontrar a melhor ação para o jogador MAX na raiz.

